

Unidad III: Complejidad Computacional

La clase NP

Clase 16 - Lógica para Ciencia de la Computación/Teoría de la Computación

Prof. Miguel Romero

La clase NP: motivación

¿Qué tienen en común los siguientes problemas?

$\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ fórmula proposicional satisfacible}\}.$

$\text{COMPOSITES} = \{\langle n \rangle \mid n = p \cdot q \text{ para enteros } p, q > 1\}.$

$\text{3-COL} = \{\langle G \rangle \mid G \text{ grafo no dirigido 3-coloreable}\}.$

La clase NP: motivación

Los problemas anteriores tienen la siguiente estructura:

- Dada una instancia, ¿existe una solución?
- Las soluciones tienen tamaño polinomial.
- Se puede **verificar eficientemente** si una solución dada es correcta.

La clase NP: definición

Definición:

Un lenguaje $L \subseteq \Sigma^*$ es **verificable en tiempo polinomial** si existe un **polinomio** $p: \mathbb{N} \rightarrow \mathbb{N}$ y un **algoritmo** M que toma tiempo polinomial tal que para toda palabra $w \in \Sigma^*$:

$$w \in L \iff \text{existe } x \in \Sigma^* \text{ tal que } |x| \leq p(|w|) \text{ y } M \text{ acepta } \langle w, x \rangle$$

Notación:

- x es un certificado para $w \in L$.
- M es un verificador para L .

Definición:

La clase **NP** es la clase de todos los lenguajes L que se pueden verificar en tiempo polinomial.

Ejemplos

$\langle \varphi \rangle \in \text{SAT} \iff$ existe una valuación σ tal que $\sigma(\varphi) = 1$.

El **certificado** para $\langle \varphi \rangle \in \text{SAT}$ es la valuación σ . ($|\langle \sigma \rangle| \leq |\varphi|$)

El **verificador** es la siguiente MT M :

Sobre entrada $\langle \varphi, x \rangle$:

1. Chequear que x codifica una valuación σ para φ .
2. Si $\sigma(\varphi) = 1$, **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de M : $O(|x|) + O(|\varphi|) = O(|\langle \varphi, x \rangle|)$.

$\langle \varphi \rangle \in \text{SAT} \iff$ existe x tal que $|x| \leq |\varphi|$ y M acepta $\langle \varphi, x \rangle$.

Concluimos que $\text{SAT} \in \text{NP}$.

Ejemplos

$\langle n \rangle \in \text{COMPOSITES} \iff \text{existe entero } 1 < p < n \text{ tal que } p \text{ divide } n.$

El **certificado** para $\langle n \rangle \in \text{COMPOSITES}$ es el entero p . ($|\langle p \rangle| \leq \log_2 n$)

El **verificador** es la siguiente MT M :

Sobre entrada $\langle n, x \rangle$:

1. Chequear que x codifica un entero $1 < p < n$.
2. Si p divide n , **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de M : $O(|x|) + O(\log^2 n) = O(|\langle n, x \rangle|^2)$.

$\langle n \rangle \in \text{COMPOSITES} \iff \text{existe } x \text{ tal que } |x| \leq \log_2 n \text{ y } M \text{ acepta } \langle n, x \rangle.$

Concluimos que $\text{COMPOSITES} \in \text{NP}$.

Ejemplos

$\langle G \rangle \in 3\text{-COL} \iff \text{existe un 3-coloreo } c \text{ para } G.$

El **certificado** para $\langle G \rangle \in 3\text{-COL}$ es el 3-coloreo c . ($|\langle c \rangle| \leq m$)
(m es la cantidad de nodos de G .)

El **verificador** es la siguiente MT M :

Sobre entrada $\langle G, x \rangle$:

1. Chequear que x codifica una función c de los nodos de G a $\{1, 2, 3\}$.
2. Si para toda arista (u, v) de G se cumple $c(u) \neq c(v)$, **aceptar**.
En caso contrario, **rechazar**.

Tiempo de ejecución de M : $O(|x|) + O(m^2) = O(|\langle G, x \rangle|^2)$.

$\langle G \rangle \in 3\text{-COL} \iff \text{existe } x \text{ tal que } |x| \leq m \text{ y } M \text{ acepta } \langle G, x \rangle.$

Concluimos que $3\text{-COL} \in \text{NP}$.

Ejemplos

Considere el siguiente problema:

$\text{TAUT} = \{\langle \varphi \rangle \mid \varphi \text{ fórmula proposicional que es una tautología}\}.$

¿Está TAUT en NP?

Máquinas de Turing no-deterministas

Definición:

Una **máquina de Turing no-determinista** (MTN) es una tupla

$M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$, donde:

- Q es un conjunto finito de **estados**.
- Σ es el **alfabeto de entrada**.
- Γ es el **alfabeto de la cinta**, donde $\Sigma \subseteq \Gamma$, y $\vdash, B \in \Gamma \setminus \Sigma$.
- q_0 es el **estado inicial**.
- q_f es el **estado final**.
- $\Delta \subseteq (Q \setminus \{q_f\}) \times \Gamma \times Q \times \Gamma \times \{\triangleleft, \triangleright\}$ es la **relación de transición**.

Ejecuciones de una MTN

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$ una MTN y $w \in \Sigma^*$.

Definición:

- Una **ejecución** ρ de M es una secuencia de configuraciones

$$\rho = C_0, C_1, C_2, \dots \text{ (no necesariamente finita)}$$

tal que $C_i \xrightarrow{M} C_{i+1}$ para todo $i \geq 0$.

- Decimos que $\rho = C_0, C_1, C_2, \dots$ es **una ejecución** de M sobre w si:

- ρ es una ejecución de M .
- $C_0 = \vdash q_0 w$. (configuración inicial de M sobre w)
- Si $\rho = C_0, \dots, C_m$ es **finito**, entonces C_m es de **detención**.

Ahora es posible tener **varias** ejecuciones de M sobre w .

Lenguaje de una MTN

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$ una MTN y $w \in \Sigma^*$.

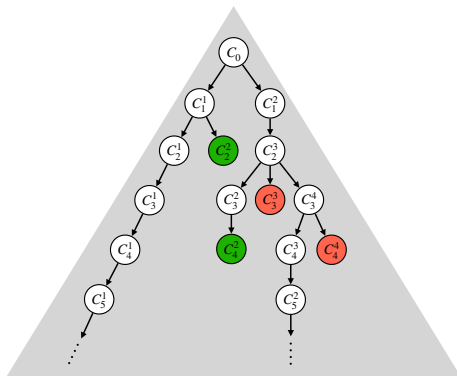
Definición:

- M **acepta** w si **existe** una ejecución ρ de M sobre w que cumple:
 - $\rho = C_0, \dots, C_m$ es finita. (M se detiene con w)
 - C_m es una configuración de aceptación.
 - Decimos que ρ es una **ejecución de aceptación** para M y w .
- El **lenguaje aceptado por** M se define como:

$$L(M) = \{w \in \Sigma^* \mid M \text{ acepta } w\}.$$

Visualizando MTNs

Podemos visualizar las ejecuciones de M sobre w con el **árbol de configuraciones** $\mathcal{T}_{M,w}$.



- Los nodos son configuraciones. La raíz es la configuración inicial.
- Los hijos de un nodo C son las siguientes configuraciones de C .
- Las hojas son configuraciones de detención (**aceptación** o **rechazo**).

Visualizando MTNs

Podemos visualizar las ejecuciones de M sobre w con el **árbol de configuraciones** $\mathcal{T}_{M,w}$.

M acepta $w \iff$ existe una **ejecución de aceptación** para M y w
 $\iff$ existe una **rama** en $\mathcal{T}_{M,w}$
que termina en una **configuración de aceptación**

Otra forma útil de pensar una MTN M :

- Dada una entrada $w \in \Sigma^*$, comenzamos en la configuración inicial.
- Si M se encuentra con más de una opción de transición, escoge sólo una de las transiciones.
- El objetivo de M es “adivinar” una **ejecución de aceptación**.
- El superpoder de M es tener **mucha suerte**:
si existe una ejecución de aceptación, M adivina correctamente.

Usando MTNs

Las MTNs sirven para chequear existencia de soluciones:

- Dada una instancia w , ¿existe una solución para w ?
- Con una MTN podemos adivinar una solución para w .
(en caso que exista.)

Ejemplo:

La siguiente MTN M acepta SAT:

Sobre entrada $\langle \varphi \rangle$:

1. Adivinar una valuación σ para φ .
2. Si $\sigma(\varphi) = 1$, **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de una MTN

Sea una MTN M que para toda entrada, **todas** sus ejecuciones son finitas.

Definición:

- Sea una palabra $w \in \Sigma^*$ y $\rho = C_0, \dots, C_m$ una ejecución de M con w . Definimos la **cantidad de pasos** $p(\rho)$ de la ejecución ρ como m .

- Sea una palabra $w \in \Sigma^*$. Definimos $p_M(w)$ como:

$$p_M(w) = \max\{p(\rho) \mid \rho \text{ ejecución de } M \text{ con } w\}.$$

- El **tiempo de ejecución (de peor caso)** de M es $t_M : \mathbb{N} \rightarrow \mathbb{N}$ tal que:

$$t_M(n) = \max\{p_M(w) \mid w \in \Sigma^* \text{ y } |w| = n\}.$$

Comentarios:

- Decimos que M **toma tiempo** t_M .
- Si M toma tiempo $O(f(n))$ entonces:

Para toda palabra w de largo n y toda posible ejecución de M con w , la cantidad de pasos es $\leq c \cdot f(n)$. (para $n \geq n_0$.)

Complejidad no-determinista de un lenguaje y NTIME

Definición:

Sea una función $f : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$.

- Un lenguaje L se puede resolver de manera no-determinista en tiempo $O(f(n))$, si existe una MTN M tal que $L = L(M)$ y $t_M(n) = O(f(n))$.
- La clase de complejidad $\text{NTIME}(f(n))$ es la clase de todos los lenguajes L que pueden ser resueltos de manera no-determinista en tiempo $O(f(n))$.

Comentarios:

- Si $f(n) = O(g(n))$, entonces $\text{NTIME}(f(n)) \subseteq \text{NTIME}(g(n))$.
- $\text{DTIME}(f(n)) \subseteq \text{NTIME}(f(n))$, para toda función f .

Teorema:

$$\text{NP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

Observaciones:

- $L \in \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k) \iff L$ se puede resolver de manera no-determinista en tiempo $O(n^k)$, para algún $k \in \mathbb{N}$.
- Decimos que L se puede resolver de manera no-determinista en **tiempo polinomial**.

Ejemplos

Veamos que **SAT** se puede resolver en tiempo polinomial, de manera no-determinista.

Podemos tomar la siguiente **MTN** M :

Sobre entrada $\langle \varphi \rangle$:

1. Adivinar una valuación σ para φ .
2. Si $\sigma(\varphi) = 1$, **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de M : $O(|\varphi|) + O(|\varphi|) = O(|\varphi|)$.

Ejemplos

Veamos que **COMPOSITES** se puede resolver en tiempo polinomial, de manera no-determinista.

Podemos tomar la siguiente **MTN** M :

Sobre entrada $\langle n \rangle$:

1. Adivinar un entero $1 < p < n$.
2. Si p divide a n , **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de M : $O(\log_2 n) + O(\log_2^2 n) = O(\log_2^2 n)$.

Ejemplos

Veamos que 3-COL se puede resolver en tiempo polinomial, de manera no-determinista.

Podemos tomar la siguiente MTN M :

Sobre entrada $\langle G \rangle$:

1. Adivinar una función c de los nodos de G a $\{1, 2, 3\}$.
2. Si para toda arista (u, v) de G se cumple $c(u) \neq c(v)$, **aceptar**.
En caso contrario, **rechazar**.

Tiempo de ejecución de M : $O(m) + O(m^2) = O(m^2)$.

Sea $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta)$ una MTN y $w \in \Sigma^*$.

Definimos:

- $\Delta(q, a) = \{(q', b, D) \in Q \times \Gamma \times \{\triangleleft, \triangleright\} \mid (q, a, q', b, D) \in \Delta\}.$
- $m = \max\{|\Delta(q, a)| \mid q \in Q, a \in \Gamma\}.$

Asumimos que los triples de $\Delta(q, a)$ están enumerados (comenzando de 1).

Representamos cada rama del árbol de configuraciones $\mathcal{T}_{M,w}$ como una palabra en $\{1, \dots, m\}^*$. (secuencia de direcciones)

Recordatorio: de MTNs a MTDs

Dado: $M = (Q, \Sigma, \Gamma, q_0, q_f, \Delta_M)$ una MTN.

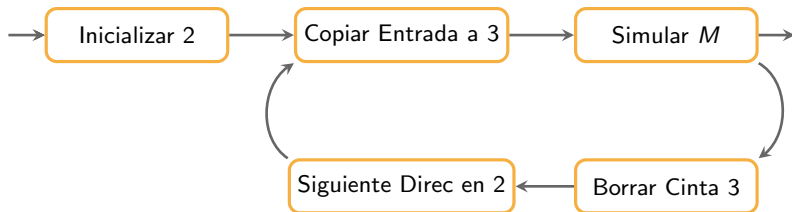
La MTD simuladora S tiene 3 cintas tal que:

- **Cinta 1:** mantiene la **palabra de entrada** w sin modificarla.
- **Cinta 2:** secuencia de **direcciones** que indica qué transiciones tomar.
- **Cinta 3:** **simulación** de M según las direcciones en Cinta 2.

Actualizamos la secuencia de direcciones de la **Cinta 2** según el orden lexicográfico $\leq$.

Recordatorio: de MTNs a MTDs

Estructura de la MTD S :



NP y no-determinismo

Teorema:

$$\text{NP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

Demostración ($\Rightarrow$):

Sea L tal que existe polinomio p y MTD polinomial M tal que para todo w :

$$w \in L \iff \text{existe } x \text{ tal que } |x| \leq p(|w|) \text{ y } M \text{ acepta } \langle w, x \rangle.$$

La siguiente MTN N resuelve L :

Sobre entrada w :

1. Adivinar una palabra x con $|x| \leq p(|w|)$.
2. Simular $M(w, x)$. Si M acepta, **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de N : $O(p(|w|)) + O(q(|w| + p(|w|)))$

(q es el polinomio que acota el tiempo de M)

NP y no-determinismo

Teorema:

$$\text{NP} = \bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$$

Demostración ($\Leftarrow$):

Sea L tal que puede ser resuelto por una MTN N que toma tiempo polinomial p .

Tomemos el siguiente verificador M :

Sobre entrada $\langle w, x \rangle$:

1. Chequea que x sea una secuencia de direcciones u con $|u| \leq p(|w|)$.
2. Simular M con w , siguiendo las direcciones u .

Si M acepta, **aceptar**. En caso contrario, **rechazar**.

Tiempo de ejecución de M : $O(|x|) + O(p(|w|)) = O(|\langle w, x \rangle| + p(|\langle w, x \rangle|))$

Se cumple para todo w :

$w \in L \iff$ existe una secuencia de direcciones u tal que M acepta w al seguir u .

$\iff$ existe x tal que $|x| \leq p(|w|)$ y M acepta $\langle w, x \rangle$.

NP y EXP

Observación: $P \subseteq NP$. (¿por qué?)

Teorema:

Sea $t : \mathbb{N} \rightarrow \mathbb{N}$. Toda MTN N que toma tiempo $t(n)$, puede ser simulada por una MTD con 3 cintas que toma tiempo $2^{O(t(n))}$.

Demostración:

Propuesta. (revisar la simulación de MTNs a MTDs.)

Consecuencia: $NP \subseteq EXP$.

Tenemos:

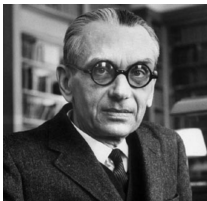
$$P \subseteq NP \subseteq EXP$$

(se sabe que $P \subsetneq EXP$.)

$\zeta P \neq NP?$

"If $P=NP$, then the world would be a profoundly different place than we usually assume it to be. There would be no special value in "creative leaps," no fundamental gap between solving a problem and recognizing the solution once it's found. Everyone who could appreciate a symphony would be Mozart; everyone who could follow a step-by-step argument would be Gauss..."

(Scott Aaronson)



Kurt Gödel (1906 - 1978)



John Von Neumann (1903 - 1957)

"... One can obviously easily construct a Turing machine, which for every formula F in first order predicate logic and every natural number n , allows one to decide if there is a proof of F of length n (length = number of symbols). Let $\Psi(F, n)$ be the number of steps the machine requires for this and let $\phi(n) = \max_F \Psi(F, n)$. The question is how fast $\phi(n)$ grows for an optimal machine..... If there really were a machine with $\phi(n) \sim k \cdot n$ (or even $\sim k \cdot n^2$), this would have consequences of the greatest importance. Namely, it would obviously mean that in spite of the undecidability of the Entscheidungsproblem, the mental work of a mathematician concerning Yes-or-No questions could be completely replaced by a machine. After all, one would simply have to choose the natural number n so large that when the machine does not deliver a result, it makes no sense to think more about the problem. Now it seems to me, however, to be completely within the realm of possibility that $\phi(n)$ grows that slowly... "

(Carta pérdida de Gödel a Von Neumann)

$\zeta P \neq NP?$